Planning Report for a Generic Auction Simulator

Programmed and LLM Based Agents

Code	HAN2601	**Supervisor**	Prof. ZHANG Hanrui
Students	LIU Xianlong and MA Ho Tsun Matthew	**Submission**	30 September 2026

Project Goal

The project will develop a web-based auction simulator in which an auctioneer and multiple bidders communicate through a common message protocol. The complete platform is intended to support both programmed agents and large language model agents. The first version will deliberately use only a fixed scripted auctioneer and scripted bidders. This narrower scope will allow the team to verify message delivery, synchronous turn handling, auction rules, allocation, payment and profit calculation before adding the uncertainty and cost of LLM calls.

Significance of the Project

What. The project provides a reusable environment for running and observing auctions under controlled rules and valuations. **Who.** Students and researchers can use it to compare bidding strategies, inspect complete auction histories and later evaluate LLM agents under the same conditions as programmed baselines. **Why.** Existing experiments often combine communication, auction logic and strategy in one implementation, which makes failures difficult to isolate and results difficult to reproduce. A common receiver and message format will separate these concerns and create a reliable baseline for later intelligent agents.

Problem Statement

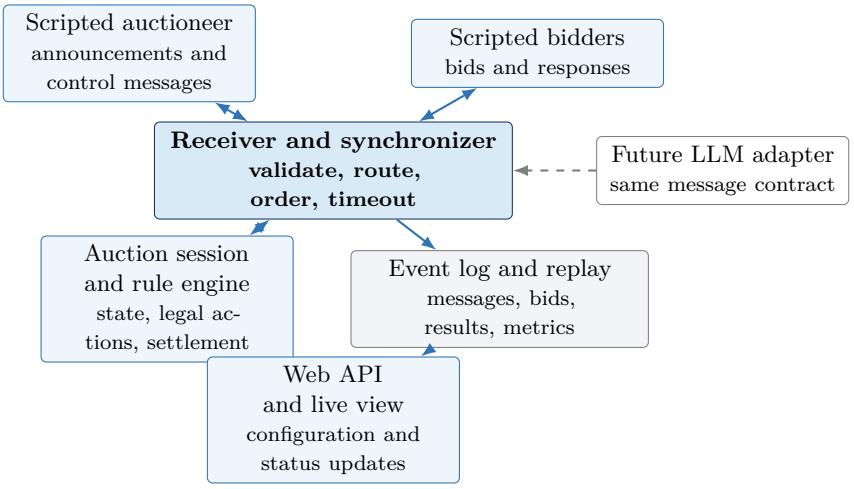
An auction involves messages from both the auctioneer and bidders, but those messages must be processed in the correct session, round and order. Without central coordination, two bidders may act at the same time, an out-of-date bid may be accepted, or one agent may see information that should remain private. The backend must therefore receive every message, verify its sender and context, route it to the correct participant, and synchronize the next action. It must also keep the auction state and event history consistent if an agent sends an invalid message or fails to respond.

Proposed Solution and Deliverables

- **Receiver and synchronizer.** A central backend component will accept messages from the auctioneer and bidders, validate their envelopes, control the current turn or round, route permitted information and handle response deadlines.
- **Common message protocol.** Messages will contain a session identifier, sender, recipient or audience, message type, round number and structured payload. The same protocol can later carry LLM-generated actions.
- **Deterministic auction service.** Coded rules will maintain session state, decide whether an action is legal, calculate winners and payments, and remain the source of truth for each result.
- **Scripted baseline agents.** The first bidder will bid $0.8v$, where v is its assigned valuation or reference price. Winning below v produces a positive surplus. The next strategy stage will make the multiplier configurable in $[0.8, 1.2]$ to study conservative and over-bidding behaviour.
- **Event log and demonstration.** Every accepted and rejected message will be recorded for replay, debugging and performance measurement. A simple interface will show the current state, messages, bids and final result.
- **Future LLM adapter.** After the scripted version is stable, an adapter will convert receiver observations into prompts and convert structured model output back into the same action messages. LLM work is not part of the first-version acceptance criteria.

First-version success criteria. A complete scripted auction can run without manual intervention; messages are processed in a valid synchronous order; invalid or late actions cannot corrupt the session; the winner, payment and profit are correct; and the event log can reproduce the observed result.

Backend Architecture



The receiver is the communication boundary for the whole simulation. An agent never changes the auction state directly. It sends an action message to the receiver, which checks the session and round, asks the deterministic rule engine to validate and apply the action, records the outcome, and then releases the next permitted message. This design provides synchronous behaviour even if the internal components later use asynchronous network calls.

Suggested implementation. The first release can be a modular Python backend with a REST API and a WebSocket event stream. The receiver, auction rules, scripted strategies and event store should be separate modules inside one deployable application. SQLite is sufficient for the prototype; a larger database or worker queue should be added only if experiment volume requires it.

Proposed Timeline

Period	Work package	Expected output
24 Sep–11 Oct 2026	Protocol and architecture	Message schema, receiver interfaces, session states and automated test plan
12 Oct–8 Nov 2026	Scripted vertical slice	Fixed auctioneer, $0.8v$ bidder, synchronous receiver and one complete auction
9 Nov–6 Dec 2026	Auction rules	Required sealed-bid mechanisms, validation, settlement and invariant tests
7 Dec 2026–17 Jan 2027	Strategy experiments	Configurable multiplier in $[0.8, 1.2]$, repeatable scenarios and profit metrics
18 Jan–21 Feb 2027	Dynamic auctions and storage	English, Dutch and Japanese flows, event log, replay and export
22 Feb–31 Mar 2027	Demonstration and testing	REST/WebSocket API, live view, failure tests and documentation
1 Apr 2027–final deadline	Evaluation and submission	Comparative experiments, analysis, demonstration and final report; optional LLM adapter if the baseline is stable

Reference. J. Chen et al., "Put Your Money Where Your Mouth Is: Evaluating Strategic Planning and Execution of LLM Agents in an Auction Arena," arXiv:2310.05746, 2024. Project page: `https://auction-arena.github.io/`.